

Module 5 | Solving Linear Systems using Iterative Methods



This chapter is concerned with techniques for solving a linear system of the form

$$A\vec{x} = \vec{b} \quad (5.1)$$

where A is an $n \times n$ square matrix with elements a_{ij} , and $\vec{x}, \vec{b}$ are $n \times 1$ vectors. We will focus on solving equation (5.1) both *accurately* and *efficiently*.

Note: Although this seems like a conceptually easy problem (just use Gaussian elimination!), it is actually a hard one when n gets large. Nowadays, linear systems with $n = 1$ million arise routinely in computational problems.

You are most likely familiar with the Gaussian elimination method, for solving linear systems $A\vec{x} = \vec{b}$, which converts the systems via elementary row operations until the solutions are easy to deduce. These type of methods have the property of producing the exact answer (in exact arithmetic) in a predictable (finite) number of operations. However, the number of operations can approach astronomical proportions. And, some zero entries in the original matrix (which don't require storage or processing) may be replaced by non-zero entries as the method proceeds; necessitating more storage and processing at subsequent steps.

DIGRESSION

Since elimination methods take a finite number of steps to solve a system of linear equations, they are often compared by counting the individual operations needed to solve a general $n \times n$ system (n equations for n unknowns). In general, the amount of time required to perform a multiplication or division on a computer is approximately the same and is considerably greater than that required to perform an addition or subtraction. So we have a couple of different ways of computing the number of operations.

- (a) total individual operations (multiplications/divisions and additions/subtractions of numbers).
- (b) total individual multiplications/divisions only
- (c) total individual additions/subtractions only

We do not count additions/subtractions or multiplications/divisions of zeroes created along the steps of the process.

The Gaussian elimination algorithm for a $n \times n$ matrix requires $\mathcal{O}(n^3)$ operations. (If you are unfamiliar with the big-O notation, we will talk about it later.)

Below, we'll consider a new kind of solution method for linear systems that avoids this problem by maintaining the original matrix structure.

§5.1 Iterative Methods

The basic idea behind *iterative methods* for solving linear systems is to start with an initial approximation $\vec{x}^{(0)}$ to the solution $\vec{x}$ and generate a sequence of vectors

$$\{\vec{x}^{(0)}, \vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(k)}, \dots\}$$

that converges to $\vec{x}$. To generate the sequence, we'll use some iteration affine mapping

$$g(\vec{x}) = T\vec{x} + \vec{c}$$

where T is a (square) matrix and $\vec{c}$ is a vector (of the same dimension as $\vec{x}$) satisfying

$$\vec{x} = g(\vec{x}) \iff A\vec{x} = \vec{b} \quad (5.2)$$

The iteration is

$$\vec{x}^{(k+1)} = T\vec{x}^{(k)} + \vec{c}, \quad k = 0, 1, 2, \dots$$

and we stop when some computable proxy for “small error” is satisfied. Two common choices are:

(a) **Relative update:**

$$\frac{\|\vec{x}^{(k+1)} - \vec{x}^{(k)}\|_*}{\|\vec{x}^{(k+1)}\|_*} \leq \epsilon.$$

(b) **Relative residual:** define $\vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)}$ and require

$$\frac{\|\vec{r}^{(k)}\|_*}{\|\vec{b}\|_*} \leq \epsilon.$$

The residual criterion is usually closer to what we actually want (small defect in $A\vec{x} = \vec{b}$). A small update does not automatically imply a small residual, especially near stagnation or if the problem is ill-conditioned.

Note: At this point, you might ask why should the sequence $\vec{x}^{(0)}, \vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(k)}, \dots$ converge? In fact, it may not always converge. We will come back to this question at the end of this section.

5.1.1 Matrix splittings

To construct T and $\vec{c}$, we use a *splitting* of A :

$$A = M - N,$$

where M is chosen to be nonsingular and easy to solve with. Substituting into $A\vec{x} = \vec{b}$ gives

$$M\vec{x} = N\vec{x} + \vec{b} \iff \vec{x} = M^{-1}N\vec{x} + M^{-1}\vec{b}.$$

Thus we take

$$T = M^{-1}N, \quad \vec{c} = M^{-1}\vec{b}.$$

5.1.2 The Jacobi Method and The Gauss-Seidel Method

Jacobi. The following version of an iteration method was named after the German mathematician Carl Gustav Jacob Jacobi who proposed a (more complicated) variant of it in the mid-1800's. **The Jacobi method** uses the matrix splitting

$$A = D - (D - A) \text{ where } M = \text{diag}(A)$$

So our iteration takes the form

$$\vec{x}^{(k+1)} = D^{-1}(D - A)\vec{x}^{(k)} + D^{-1}\vec{b} \quad (5.3)$$

Gauss-Seidel. The next iteration method we'll consider is usually credited to Gauss and another German mathematician, Philipp Ludwig von Seidel (though it is also sometimes credited to Liebmann, or simply called the method of "successive displacement"). **The Gauss-Seidel method** uses the splitting

$$A = (L + D) - (-U)$$

where $D = \text{diag}(A)$ and U and L respectively are the upper and lower triangular parts of A left over when the diagonal D is removed:

$$U = \begin{bmatrix} 0 & a_{1,2} & \cdots & a_{1,n} \\ 0 & 0 & \cdots & a_{2,n} \\ \cdot & \cdot & \cdots & \cdot \\ \cdot & \cdot & \cdots & a_{n-1,n} \\ 0 & 0 & \cdots & 0 \end{bmatrix} \quad L = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ a_{2,1} & 0 & \cdots & 0 \\ \cdot & \cdot & \cdots & \cdot \\ a_{n-1,1} & \cdot & \cdots & \cdot \\ a_{n,1} & \cdots & a_{n,n-1} & 0 \end{bmatrix} \quad (5.4)$$

In this case, the iteration takes the form

$$\vec{x}^{(k+1)} = -(L + D)^{-1}U\vec{x}^{(k)} + (L + D)^{-1}\vec{b}, \quad (5.5)$$

So, the main remaining questions in this section is to figure out whether the two iteration algorithms converge. To answer that, we first need to take a detour and talk about matrix norms and spectral radius.

§5.2 Operator Norm and Spectral Radius

We defined the operator norm of a matrix A induced by the ℓ_p -norm to be

$$\|A\|_p = \|A\|_{\ell_p \rightarrow \ell_p} = \sup_{0 \neq \vec{x} \in \mathbb{R}^n} \frac{\|A\vec{x}\|_p}{\|\vec{x}\|_p} = \sup_{\|\vec{x}\|_p=1} \|A\vec{x}\|_p$$

Theorem 5.2.32

The operator norms induced by the ℓ_1 -norm and ℓ_∞ -norm satisfy

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|, \quad (\text{maximum column sum})$$

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|. \quad (\text{maximum row sum})$$

Proof of theorem 32.

We will prove the result for the ℓ_1 -norm first. Starting from the definition of the ℓ_1 vector norm, we have

$$\|A\vec{x}\|_1 = \sum_{i=1}^n \left| \sum_{j=1}^n a_{ij}x_j \right| \leq \sum_{i=1}^n \sum_{j=1}^n |a_{ij}| |x_j| = \sum_{j=1}^n |x_j| \sum_{i=1}^n |a_{ij}|.$$

If we let

$$c = \max_{j=1, \dots, n} \sum_{i=1}^n |a_{ij}|$$

then

$$\|A\vec{x}\|_1 \leq c \|\vec{x}\|_1 \implies \|A\|_1 \leq c. \quad (5.6)$$

Now let m be the column where the maximum sum is attained. If we choose y to be the vector with components $y_k = \delta_{km}$, then we have $\|A\vec{y}\|_1 = c$. Since $\|\vec{y}\|_1 = 1$, we must have that

$$\max_{\|\vec{x}\|_1=1} \|A\vec{x}\|_1 \geq \|A\vec{y}\|_1 = c \implies \|A\|_1 \geq c. \quad (5.7)$$

The only way to satisfy both equation (5.6) and equation (5.7) is if $\|A\|_1 = c$. ■

■ Question 66.

Prove theorem 32 for the ℓ_∞ -norm. □

Let's get back to the iterative algorithm and analyze the formula

$$\vec{x}^{(k+1)} = T\vec{x}^{(k)} + \vec{c}$$

where $\vec{x}^{(0)}$ is arbitrary. We want our sequence of guesses $\vec{x}^{(k)}$ to converge to the exact solution $\vec{x}^*$, which is given by

$$\vec{x}^* = T\vec{x}^* + \vec{c}$$

. Let's define the error at step k as $\vec{e}^{(k)} = \vec{x}^{(k)} - \vec{x}^*$.

Then,

$$\vec{e}^{(k+1)} = \vec{x}^{(k+1)} - \vec{x}^* = (T\vec{x}^{(k)} + \vec{c}) - (T\vec{x}^* + \vec{c}) = T\vec{e}^{(k)}$$

Applying this recursively, $\vec{e}^{(k)} = T^k \vec{e}^{(0)}$. So, for the iterative method to work, the error $\vec{e}^{(k)}$ must approach the zero vector as $k \rightarrow \infty$, no matter what our initial guess $\vec{e}^{(0)}$ was.

So, our next question is whether the iteration matrices for the Jacobi and Gauss-Seidel methods satisfy $\|T\|_\infty < 1$. We show that this is true in specific cases.

Definition 5.2.33

A square matrix A is said to be *diagonally dominant* if

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i$$

where a_{ij} denotes the entry in the i th row and j th column.

One can show that a diagonally dominant matrix is nonsingular. The Gaussian elimination algorithm can be performed for such a matrix without any pivoting, and the computations remain stable with respect to the growth of round-off errors. The following exercise gives a sufficient (but not necessary) condition for the convergence of Jacobi and Gauss-Seidel algorithms.

■ Question 67.

□

Show that A is diagonally dominant iff the iteration matrix for the Jacobi method $T = D^{-1}(D - A) = I - D^{-1}A$ satisfies $\|T\|_\infty < 1$.

As an immediate corollary, we get that the Jacobi method converges if A is diagonally dominant. Moreover, by the construction of the Gauss-Seidel iteration matrix, we can show that the ratio of subsequent iterations is always bounded above by the ℓ_∞ norm of the Jacobi iteration matrix. So, we have the following result.

Theorem 5.2.34

If A is diagonally dominant, then for any choice of $\vec{x}^{(0)}$, both the Jacobi and the Gauss-Seidel methods give sequences $\{\vec{x}^{(k)}\}_{k=0}^\infty$ that converge to the unique solution of $A\vec{x} = \vec{b}$.

5.2.1 A Necessary Condition

The above theorem provides a sufficient condition for convergence. Next, let's discuss a necessary condition.

Suppose $\lambda \in \mathbb{C}$ is an eigenvalue of A with associated eigenvector $\vec{u}$. Then

$$\|A\| \geq \frac{\|A\vec{u}\|}{\|\vec{u}\|} = |\lambda|$$

Definition 5.2.35: Spectral Radius

The *spectral radius* $\rho(A)$ of A is the maximum $|\lambda|$ over all eigenvalues λ of A .

So, we have established that

$$\rho(A) \leq \|A\| \text{ for any operator norm on } A$$

In fact, we can also prove the following result (using Jordan Form or Schur Decomposition).

Theorem 5.2.36

Suppose A is any $n \times n$ matrix and let $\delta > 0$ be arbitrary. Then there is a norm N on $\mathbb{R}^n$ such that the corresponding operator norm $\|\cdot\|_N$ satisfies

$$\rho(A) \leq \|A\|_N \leq \rho(A) + \delta,$$

where $\rho(A)$ is the spectral radius.

In other words, the spectral radius is a pretty good indicator of the “size” of a matrix. In cases of iterative algorithms, we often end up with powers of matrices of the form A^k . In those cases, we are interested in precise conditions when A is a **convergent matrix**, that is, $A^k \rightarrow 0$ as $k \rightarrow \infty$. The following result says that the spectral radius can be used to ensure that our matrices are “small enough” so that the powers converge to 0.

■ **Question 68.**

Show that $A^k \rightarrow 0$ iff $\rho(A) < 1$.

As a consequence, we have the following theorem

Theorem 5.2.37

For any $\vec{x}^{(0)} \in \mathbb{R}^n$, the sequence $\{\vec{x}^{(k)}\}_{k=0}^{\infty}$ defined by

$$\vec{x}^{(k)} = T\vec{x}^{(k-1)} + \mathbf{c}, \quad \text{for each } k \geq 1,$$

converges to the unique solution of $\vec{x} = T\vec{x} + \mathbf{c}$ if and only if $\rho(T) < 1$.

§5.3 In-class Mathematica experiments

Below are some experiments designed so that we can see the phenomenon first and then explain it using iteration matrices, norms, and spectral radii.

■ **Question 69.**

Consider the system $\begin{pmatrix} 3 & 1 & -1 \\ 1 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix} \vec{x} = \begin{pmatrix} 0 \\ 5 \\ -5 \end{pmatrix}$.

- Solve the above system analytically, using methods from prior weeks.
- Why should the Jacobi method converge to the solution for the above system?
- Test your Jacobi code for the above problem, using several combinations (of your choosing) of starting vector $\vec{x}_0$ and tolerance ε , then report the results (final output and number of iterations.) Make sure that the maximum number of iterations N is sufficient to allow convergence, as it should eventually converge for all starting vectors.

■ Question 70. □

Consider the system $\begin{pmatrix} -1 & 1 & 2 \\ 6 & -1 & 5 \\ 68.5 & -28.5 & -1 \end{pmatrix} \vec{x} = \begin{pmatrix} 1 \\ 0 \\ -20.5 \end{pmatrix}$. The coefficient matrix is not invertible, and this system has infinitely many solutions, including $\begin{pmatrix} -4 \\ -9 \\ 3 \end{pmatrix}$. The spectral radius of the iteration matrix $D^{-1}(D - A)$ used in the Jacobi method is 1.

- (a) Test your Jacobi code using several combinations of starting vector $\vec{x}_0$, tolerance ε , and max number of iterations N . For each, report your inputs, whether it successfully converged, and if so, the number of iterations. If the Jacobi method failed, report the last few candidate vectors $\vec{x}_j$ before the iteration cap was reached.
- (b) Repeat part (a) with your Gauss-Seidel code, for the same combinations.
- (c) Compare the results of these two methods.

■ Question 71. □

Consider the system $\begin{pmatrix} 1 & 2 & -1 \\ 2 & 1 & 1 \\ -1 & 0 & 1 \end{pmatrix} \vec{x} = \begin{pmatrix} 3 \\ 0 \\ 3 \end{pmatrix}$, whose solution is $\begin{pmatrix} -2 \\ 3 \\ 1 \end{pmatrix}$.

- (a) Explain why the Jacobi method should fail for this system. If this requires any matrix-related computations, you don't need to do these by hand.
- (b) As in question 70.a, test your Jacobi code and report the results.
- (c) Similarly, test your Gauss-Seidel code and report the results.
- (d) Compare the methods. Though neither worked, does one appear to have performed worse than the other? How can you tell?

■ Question 72. □

Consider the system $\begin{pmatrix} 1 & \gamma - 1 \\ \gamma - 1 & 1 \end{pmatrix} \vec{x} = \begin{pmatrix} \gamma \\ \gamma \end{pmatrix}$, whose solution is $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

- (a) Find the spectral radius of the iteration matrix $D^{-1}(D - A)$ by analytic methods, e.g. via the characteristic polynomial.
- (b) Using the initial vector $\vec{x}_0 = \vec{0}$, tolerance $\varepsilon = 10^{-8}$, and a maximum number of iterations N large enough to assure convergence, run your Jacobi code for parameter values $\gamma = 2^{-2}, 2^{-4}, 2^{-6}$, and 2^{-8} . Based on the numbers of iterations required for each to converge, make a prediction for the number of iterations needed for $\gamma = 2^{-10}$, then test that prediction.